

DSA Assignment #8: 🌲 (3/3)

2500010774 兰玉琪 数学科学学院

1. 题目

M 晴问 9.7: 向下调整构建大顶堆

手搓堆, <https://sunnywhy.com/sfbj/9/7>

思路:

代码:

```
import sys
data = iter(sys.stdin.read().split())
n = int(next(data))
heap = [int(next(data)) for _ in range(n)]
def tiaozheng(i):
    '''向上调整heap[i]'''
    if i == 0:
        return
    c_val = heap[i]
    p_val = heap[(i-1)//2]
    if c_val > p_val:
        heap[i], heap[(i-1)//2] = p_val, c_val
        tiaozheng((i-1)//2)
for i in range(n):
    tiaozheng(i)
print(*heap)
```

课程训练营算法笔记题库题单比赛语言入门教程🔖机试真题NEW🔖2026考研机试高分训练营

《2026考研机试高分训练营》已经上线: <https://sunnywhy.com/camp/3415>, 难度友好型院校。

您还未设置昵称, 可点击[此处](#)进入个人中心修改

Python

第二行n个整数 a_i ($1 \leq a_i \leq 10^4$), 表示正整数序列。

输出描述

输出n个整数, 表示堆的层序序列, 中间用空格隔开, 行末不允许有多余的空格。

样例1

输入复制

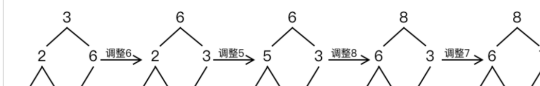
6
3 2 6 5 8 7

输出复制

8 6 7 2 5 3

解释

调整前的完全二叉树和调整后的堆如下图所示。



```
1 import sys
2 data = iter(sys.stdin.read().split())
3 n = int(next(data))
4 heap = [int(next(data)) for _ in range(n)]
5 def tiaozheng(i):
6     '''向上调整heap[i]'''
7     if i == 0:
8         return
9     c_val = heap[i]
10    p_val = heap[(i-1)//2]
11    if c_val > p_val:
12        heap[i], heap[(i-1)//2] = p_val, c_val
13        tiaozheng((i-1)//2)
14 for i in range(n):
15     tiaozheng(i)
16 print(*heap)
```

测试输入

提交结果

历史提交

完美通过

100% 数据通过测试 详情

运行时长: 0 ms

查看题

M1722.执行交换操作后的最小汉明距离

dsu, <https://leetcode.cn/problems/minimize-hamming-distance-after-swap-operations/>

思路:

代码:

```

from collections import defaultdict
class Solution:
    def minimumHammingDistance(self, source: List[int], target: List[int], allowedSwaps:
        n = len(source)
        Parent = [i for i in range(n)]
        def find(i):
            if (Parent[i] == i):
                return i
            else:
                result = find(Parent[i])
                Parent[i] = result
                return result
        def union(i,j):
            i1 = find(i)
            j1 = find(j)
            Parent[j1] = i1
        for i,j in allowedSwaps:
            union(i,j)
        Parent = [find(i) for i in range(n)]
        mydict = defaultdict(list)
        for i,item in enumerate(Parent):
            mydict[item].append(i)
        total = 0
        for value in mydict.values():
            cdict1 = defaultdict(int)
            cdict2 = defaultdict(int)
            for num in value:
                cdict1[target[num]] += 1
                cdict2[source[num]] += 1
            for key,val in cdict2.items():
                total += min(val,cdict1[key])
        return n-total

```

DescriptionEditorialSubmissionsSolutions

Status

Language

Runtime

Memory

1

Accepted

Apr 21, 2026

Python3

227 ms

63.1 MB

</> Code

Python3Auto

```
1 from collections import defaultdict
2 class Solution:
3     def minimumHammingDistance(self, source: List[int], target: List[int], allowedSwaps: List[List[int]]) -> int:
4         n = len(source)
5         Parent = [i for i in range(n)]
6         def find(i):
7             if Parent[i] == i:
8                 return i
9             else:
10                result = find(Parent[i])
```

SavedLn 32, Col 23

Testcase

Test Result

T22161: 哈夫曼编码树

greedy, <http://cs101.openjudge.cn/practice/22161/>

思路：

代码：

```

import sys
data = iter(sys.stdin.read().split())
n = int(next(data))
import heapq
class Node:
    def __init__(self,weight=0,zimu=set(),left=None,right=None):
        self.weight = weight
        self.zimu = zimu
        self.left = left
        self.right = right
    def __lt__(self,other):
        if self.weight != other.weight:
            return self.weight < other.weight
        return self.zimu < other.zimu
heap = []
mydict = {}
for _ in range(n):
    zimu = next(data)
    weight = int(next(data))
    node = Node(weight,{zimu})
    mydict[zimu] = ""
    heap.append(node)
heapq.heapify(heap)
while len(heap) >= 2:
    a = heapq.heappop(heap)
    b = heapq.heappop(heap)
    for s in a.zimu:
        mydict[s] = "0"+mydict[s]
    for s in b.zimu:
        mydict[s] = "1"+mydict[s]
    node = Node(a.weight+b.weight,a.zimu|b.zimu)
    node.left = a
    node.right = b
    heapq.heappush(heap,node)
tree = heap[0]
result = []
def jiema(num):
    i = 0
    cur = tree
    result = []
    while i <= len(num)-1:
        if num[i] == '0':
            if cur.left:

```

```

        cur = cur.left
        i += 1
    else:
        result.append(list(cur.zimu)[0])
        cur = tree.left
        i += 1
    else:
        if cur.right:
            cur = cur.right
            i += 1
        else:
            result.append(list(cur.zimu)[0])
            cur = tree.right
            i += 1
    result.append(list(cur.zimu)[0])
    return "".join(result)
while True:
    try:
        a = next(data)
        if a[0].isdigit():
            print(jiema(a))
        else:
            print("".join([mydict[s] for s in a]))
    except StopIteration:
        break

```

状态: Accepted

Source Code

```
import sys
data = iter(sys.stdin.read().split())
n = int(next(data))
import heapq
class Node:
    def __init__(self, weight=0, zimu=set(), left=None, right=None):
        self.weight = weight
        self.zimu = zimu
        self.left = left
        self.right = right
    def __lt__(self, other):
        if self.weight != other.weight:
            return self.weight < other.weight
        return self.zimu < other.zimu
heap = []
mydict = {}
for _ in range(n):
    zimu = next(data)
    weight = int(next(data))
    node = Node(weight, {zimu})
    mydict[zimu] = ""
    heap.append(node)
heapq.heapify(heap)
while len(heap) >= 2:
    a = heapq.heappop(heap)
    b = heapq.heappop(heap)
    for s in a.zimu:
        mydict[s] = "0"+mydict[s]
    for s in b.zimu:
```

基本信息

#: 51716717

题目: 22161

提交人: 25n2500010774(sleepy(25n2500010774))

内存: 3776kB

时间: 23ms

语言: Python3

提交时间: 2026-01-26 19:40:48

M 晴问 9.5: 平衡二叉树的建立

手搓 AVL, <https://sunnywhy.com/sfbj/9/5/359>

思路:

代码:

```

import sys
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.height = 1
        self.left = None
        self.right = None
def solve():
    data = sys.stdin.read().split()
    n = int(data[0])
    tree = None
    def get_height(node):
        if not node:
            return 0
        return node.height
    def rightrotate(node):
        root = node.left
        node1 = node.left.right
        root.right = node
        node.left = node1
        node.height = max(get_height(node.left), get_height(node.right))+1
        root.height = max(get_height(root.left), get_height(root.right))+1
        return root
    def leftrotate(node):
        root = node.right
        node1 = node.right.left
        root.left = node
        node.right = node1
        node.height = max(get_height(node.left), get_height(node.right))+1
        root.height = max(get_height(root.left), get_height(root.right))+1
        return root
    def insert(node, val):
        if not node:
            return TreeNode(val)
        if val < node.val:
            node.left = insert(node.left, val)
        else:
            node.right = insert(node.right, val)
        node.height = max(get_height(node.left), get_height(node.right))+1
        balance = get_height(node.left)-get_height(node.right)
        if balance > 1:
            if val < node.left.val:
                node = rightrotate(node)

```



```

        else:
            node.left = leftrotate(node.left)
            node = rightrotate(node)
    elif balance < -1:
        if val > node.right.val:
            node = leftrotate(node)
        else:
            node.right = rightrotate(node.right)
            node = leftrotate(node)

    return node
for i in range(1,n+1):
    tree = insert(tree,int(data[i]))
result = []
def preorder(node):
    if node:
        result.append(node.val)
        preorder(node.left)
        preorder(node.right)
preorder(tree)
print(*result)
if __name__ == "__main__":
    solve()

```

 alt text

M208.实现 Trie（前缀树）

trie, <https://leetcode.cn/problems/implement-trie-prefix-tree/>

思路：

代码

```

class Trie:
    def __init__(self):
        self.tree = {}
    def insert(self, word: str) -> None:
        cur = self.tree
        for i in range(len(word)-1):
            a = word[i]
            if a in cur:
                cur = cur[a]
            else:
                cur[a] = {}
                cur = cur[a]
        if word[-1] in cur:
            cur = cur[word[-1]]
            cur[None] = {}
        else:
            cur[word[-1]] = {None:{}}
    def search(self, word: str) -> bool:
        cur = self.tree
        for t in list(word):
            if t not in cur:
                return False
            cur = cur[t]
        if None in cur:
            return True
        return False
    def startsWith(self, prefix: str) -> bool:
        cur = self.tree
        for t in list(prefix):
            if t not in cur:
                return False
            cur = cur[t]
        return True

```

```

# Your Trie object will be instantiated and called as such:
# obj = Trie()
# obj.insert(word)
# param_2 = obj.search(word)
# param_3 = obj.startsWith(prefix)

```

Description

Editorial

Submissions

Solutions

Status

Language

Runtime

Memory

Accepted

Nov 28, 2025

Python3

37 ms

30.7 MB

Code

Accepted

All Submissions

3ms

23ms

43ms

63ms

Code | Python3

```
1 class Trie:
2     def __init__(self):
3         self.tree = {}
4     def insert(self, word: str) -> None:
5         cur = self.tree
6         for i in range(len(word)-1):
7             a = word[i]
8             if a in cur:
9                 cur = cur[a]
10            else:
```

Testcase

Test Result

You must run your code first

M307. 区域和检索 - 数组可修改

segment tree, <https://leetcode.cn/problems/range-sum-query-mutable/>

思路：

法一树状数组（感谢老师的反复训练，现在手搓这个已经没有心梗了）

法二线段树

代码

```

from itertools import accumulate
class NumArray:
    def __init__(self, nums):
        self.nums = nums
        self.shuzhuangshuzu = [0 for _ in range(len(nums)+1)]
        qianzhuihe = [0]+list(accumulate(self.nums))
        for i in range(1, len(nums)+1):
            lowbit = i&(-i)
            self.shuzhuangshuzu[i] = qianzhuihe[i]-qianzhuihe[i-lowbit]
    def update(self, index: int, val: int) -> None:
        a = val-self.nums[index]
        self.nums[index] = val
        index += 1
        while index <= len(self.nums):
            self.shuzhuangshuzu[index] += a
            index += (index&(-index))
    def sumRange(self, left: int, right: int) -> int:
        def calsum(i):
            total = 0
            while i:
                total += self.shuzhuangshuzu[i]
                i -= (i&(-i))
            return total
        return calsum(right+1)-calsum(left)

```

```

import math
class NumArray:
    def __init__(self, nums: List[int]):
        n = 2**math.ceil(math.log2(len(nums)))
        self.tree = [0]*(2*n)
        for i in range(len(nums)):
            self.tree[n+i] = nums[i]
        for i in range(n-1,0,-1):
            self.tree[i] = self.tree[2*i] + self.tree[2*i+1]
    def update(self, index: int, val: int) -> None:
        index = index+(len(self.tree)//2)
        difference = val-self.tree[index]
        while index != 0:
            self.tree[index] += difference
            index //= 2
    def sumRange(self, left: int, right: int) -> int:
        left += len(self.tree)//2
        right += len(self.tree)//2
        right += 1
        total = 0
        while left<right:
            if left % 2 != 0:
                total += self.tree[left]
                left += 1
            if right % 2 != 0:
                right -= 1
                total += self.tree[right]
            left //= 2
            right //= 2
        return total

```

The screenshot shows a coding competition interface. On the left, a table displays submission results:

	Status	Language	Runtime	Memory
4	Accepted a few seconds ago	Python3	460 ms	38 MB
3	Accepted Mar 11, 2026	Python3	284 ms	38.2 MB

On the right, the 'Code' editor shows the following Python code:

```

20 total = 0
21 while left<right:
22     if left % 2 != 0:
23         total += self.tree[left]
24         left += 1

```

2. 学习总结和个人收获

对老师上课讲的fancy的一些技巧在纸上implement一下加深了理解，写起代码来越发行云流水